

ECS Game Engine

Custom entity-component-system engine with a job-based scheduler, hot-reloadable gameplay modules, and a Vulkan forward+ renderer.

C++	Vulkan	ECS	CMake
-----	--------	-----	-------

VERSION	STATUS	LICENSE	LANGUAGE	GRAPHICS	UPDATED
0.5.0	Open Source	Zlib	C++20	Vulkan 1.3	Sep 2026

1. Overview

A custom engine built around an archetype entity-component-system, a job scheduler that derives parallelism from declared component access, and gameplay modules that can be rebuilt and reloaded while the game runs. Rendering is a Vulkan forward+ path with clustered light culling, sharing the same job system as simulation rather than owning a thread of its own.

The design bet is that declaring what a system reads and writes is enough to schedule a frame correctly. Systems never take locks and never spawn their own threads; the scheduler builds a dependency graph from access declarations each frame and runs everything that can run at once. A data race in gameplay code is therefore a declaration bug, catchable at startup rather than at 3 a.m. in a crash report.

2. Frame structure

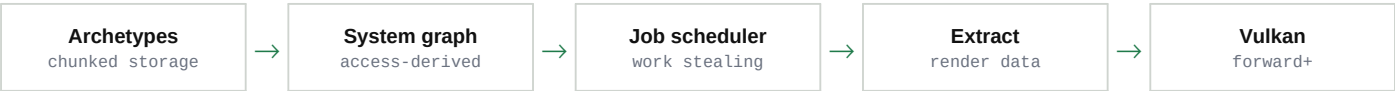


Figure 1 — Extraction copies render-relevant components into a frame packet, so rendering never reads live simulation state.

Entities live in 16 KB chunks grouped by archetype, so iterating a query is a linear walk over packed component arrays with no indirection per entity. Structural changes — adding or removing components — are recorded into command buffers and applied at a sync point between system stages, which keeps iteration safe without invalidating the chunk layout mid-pass.

3. Subsystems

ECS core	Archetype chunk storage, typed queries, deferred structural change command buffers.
Scheduler	Builds a per-frame dependency graph from read/write sets; work-stealing job pool.
Module host	Gameplay compiled to shared libraries; reload swaps code and migrates component data.
Renderer	Forward+ with clustered light culling, bindless descriptors, and a frame graph for passes.
Asset system	Content-hashed assets with async streaming and hot reload of textures, meshes, shaders.
Build	CMake presets for Windows, Linux, and macOS via MoltenVK; no external package manager.

4. System declaration

```
struct MovementSystem {
    // Access declarations drive scheduling; no locks anywhere in user code.
    using Reads  = ecs::Access<Velocity, const DeltaTime>;
    using Writes = ecs::Access<Transform>;

    void run(ecs::Query<Transform&, const Velocity&> q, const DeltaTime& dt) {
        q.par_for_each([&](Transform& t, const Velocity& v) {
            t.position += v.linear * dt.seconds;    // chunk-parallel, no sync
        });
    }
};

world.add_system<MovementSystem>(Stage::Simulation);
world.add_system<CollisionSystem>(Stage::Simulation).after<MovementSystem>();
world.add_system<ExtractRenderables>(Stage::Extract);
```

Listing 1 — Conflicting declarations are detected when the graph is built, not at runtime.

5. Hot reload

- **What reloads.** Gameplay modules build to shared libraries. On change the host unloads the old library, loads the new one, and re-registers its systems within the existing world.
- **State migration.** Component layouts are versioned and reflected; fields that still match by name and type carry over, new fields take defaults, and removed fields are dropped with a log line naming each one.
- **Limits.** Engine-side types and the renderer do not reload; a change there needs a restart. Reload is a five-second edit loop for gameplay, not a general substitute for building.

6. Performance

SCENARIO	FRAME TIME	NOTES
100k entities, 6 systems	2.4 ms	Simulation only, 8 worker threads.
1M entities, transform only	7.9 ms	Chunk-parallel iteration, no structural changes.
Forward+, 1,200 lights	4.6 ms	1440p, clustered culling, RTX 4060.
Module reload	620 ms	Rebuild excluded; swap and migration only.

7. Scope and constraints

- There is no editor. Scenes are defined in code or loaded from data files, which suits the engine's intended use as a foundation for programmer-driven projects.
- Vulkan 1.3 with dynamic rendering and bindless descriptors is required; there is no fallback path for older drivers, and macOS runs through MoltenVK with known validation-layer noise.
- Physics, audio, and networking are deliberately out of scope. The engine provides the ECS, the scheduler, and the renderer; everything else is a module.

8. Roadmap

RELEASE	PLANNED WORK	TARGET
0.6	Frame graph resource aliasing and async compute queue support.	Q4 2026
0.7	Relationship components for hierarchies and entity references.	Q1 2027
1.0	Stable module ABI, serialization format, and API documentation.	Q2 2027